⚙️ CATEGORY 2: Process Management — Complete Deep Dive

2.1 Process Basics — PID, PPID, `ps`, `pstree`, Foreground/Background

🔷 What a process is in simple terms

A process is a **running instance of a program**. When you execute a binary, the kernel loads it into memory, assigns it a unique Process ID (PID), and starts executing its instructions. Every process on Linux descends from a single ancestor — `init` (or `systemd`), PID 1.

🔷 How processes are created internally

```
fork() + exec() — the UNIX process creation model

Parent process                    Child process
      │                                │
      │── fork() ──────────────────────►│  Exact copy of parent
      │             (copy-on-write)     │  (same memory, FDs, env)
      │                                │
      │                                │── exec() ──► loads new program
      │                                │              replaces memory
image
      │◄── wait() ─────────────────────│  Parent waits (or not)
      │    (collects exit status)       │

Every shell command you run:
  bash → fork() → child bash → exec("/usr/bin/ls") → ls runs
```

🔷 PID and PPID

```
# PID  = Process ID     — unique identifier for THIS process
# PPID = Parent PID     — who created this process
# UID  = User ID        — who owns this process
# GID  = Group ID       — group ownership

# Special PIDs:
# PID 0 = swapper/idle (kernel, not visible in ps)
# PID 1 = init / systemd (ancestor of ALL user processes)
# PID 2 = kthreadd (ancestor of ALL kernel threads)
```

```
# Your current shell's PID
echo $$          # PID of current shell
echo $PPID       # PID of parent (the terminal or SSH session)
echo $!          # PID of last background process

# if parent dies before child → child is "orphaned" → adopted by PID 1
```

🔷 ps — Process snapshot

```
# ── Two syntax styles ──────────────────────────────────────────
# BSD style (no dash):    ps aux
# UNIX style (with dash): ps -ef

# ── ps aux — the most used ─────────────────────────────────────
ps aux
# USER     PID  %CPU %MEM    VSZ    RSS  TTY    STAT START  TIME COMMAND
# root       1   0.0  0.1  16952   5432  ?      Ss   10:00  0:02 /sbin/init
# www-data 1234  2.5  3.2 512334 131072  ?      Sl   10:05  1:23 nginx:
worker
# alice    5678  0.1  0.4  98765  16384  pts/0 S+   11:30  0:00 bash

# Column meanings:
# USER    → process owner
# PID     → process ID
# %CPU    → CPU usage (averaged since process start)
# %MEM    → percentage of physical RAM used
# VSZ     → Virtual memory size (KB) — all mapped memory
# RSS     → Resident Set Size (KB) — physical RAM actually in use NOW
# TTY     → terminal (? = no terminal / daemon)
# STAT    → process state (R=running, S=sleeping, D=disk wait, Z=zombie)
# START   → when process started
# TIME    → cumulative CPU time consumed
# COMMAND → command line

# ── ps -ef — UNIX style (shows PPID) ───────────────────────────
ps -ef
# UID   PID  PPID  C  STIME  TTY   TIME    CMD
# root    1     0  0  10:00  ?     0:02    /sbin/init
# root    2     0  0  10:00  ?     0:00    [kthreadd]
# PPID column is the KEY advantage of -ef over aux

# ── Custom output format — most powerful ───────────────────────
ps -eo pid,ppid,user,stat,pcpu,pmem,comm --sort=-pcpu | head -20
# -eo          = select specific columns for ALL processes
# --sort=-pcpu = sort by CPU descending (- = descending)

# Find top CPU consumers
ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head -10
```

```
# Find top memory consumers
ps -eo pid,ppid,cmd,%mem,rss --sort=-rss | head -10

# Show process tree
ps aux --forest       # ASCII tree showing parent-child relationships

# Show threads of a process
ps -p 1234 -L         # Threads of PID 1234

# Check if process is running in a script
if ps -p "$PID" > /dev/null 2>&1; then
    echo "Process $PID is running"
fi
# By name:
if pgrep -x "nginx" > /dev/null; then
    echo "nginx is running"
fi

# Full command without truncation
ps auxww                                  # ww = wide, no truncation
cat /proc/1234/cmdline | tr '\0' ' '      # Definitive source from kernel
```

🔷 `pstree` — Visual process hierarchy

```
pstree
# systemd─┬─accounts-daemon───2*[{accounts-daemon}]
#         ├─dockerd─┬─containerd───10*[{containerd}]
#         ├─nginx───4*[nginx]
#         ├─sshd───sshd───bash───pstree
#         └─systemd-journal

pstree -p        # Show PIDs
pstree -u        # Show username changes
pstree 1234      # Tree rooted at PID 1234
pstree alice     # All processes owned by alice
pstree -pa       # Show PIDs and full arguments

# Why pstree matters in production:
# - Shows parent-child chains ps hides
# - Understand why kill doesn't propagate
# - Debug container process hierarchies
# - Find orphaned process chains
```

🔷 Foreground vs Background

```
# Foreground (default): shell blocks, Ctrl+C kills, Ctrl+Z suspends
sleep 100
```

```
# Background (&): shell returns immediately
sleep 100 &     # [1] 12345

# Multiple background jobs
make build &    # [1] 12345
make test &     # [2] 12346

# Wait for background jobs
wait            # Wait for ALL
wait 12345      # Wait for specific PID
wait %1         # Wait for job number 1

# TTY: ? = no terminal (daemon), pts/0 = SSH/xterm, tty1 = physical
console
```

🔷 Short crisp interview answer

> "Every process has a PID and PPID. Processes are created via `fork()` which clones the parent, then `exec()` loads the new binary image. `ps aux` gives a snapshot — I pay attention to the STAT column, RSS for actual RAM use, and `--sort=-%cpu` to find CPU hogs. `ps -ef` is useful specifically because it shows PPID, letting me trace parent-child chains. `pstree -p` visualizes the whole hierarchy. `&` runs in background, Ctrl+Z suspends, and the shell tracks these as numbered jobs."

⚠ Gotchas

```
# GOTCHA 1: VSZ vs RSS
# VSZ = virtual address space (includes shared libs, mmap gaps) —
MISLEADING
# RSS = actual physical RAM pages in use — the real number
# VSZ >> RSS is NORMAL. Watch RSS for memory pressure.

# GOTCHA 2: %CPU is averaged since process start, not instantaneous
# Use top for real-time CPU percentages.

# GOTCHA 3: Killing a parent doesn't always kill children
# Children become orphans → adopted by PID 1 → keep running!
# To kill a process group: kill -- -PGID (negative = process group)

# GOTCHA 4: [brackets] in ps = kernel threads — don't kill them
# [kworker/0:0], [migration/0] = kernel internals

# GOTCHA 5: ps truncates long command lines
ps auxww                                    # ww = disable truncation
cat /proc/PID/cmdline | tr '\0' ' '         # Full command from kernel
```

2.2 Job Control — `fg`, `bg`, `jobs`, `&`, `nohup`, `disown`

🔷 What job control is

Job control lets you **manage multiple processes from a single shell session** — suspending, resuming, and moving processes between foreground and background. It's the shell's built-in task manager, built on POSIX signal semantics.

🔷 How job control works internally

```
Terminal (pts/0)
      ├── SIGINT  (Ctrl+C)   → kills foreground process GROUP
      ├── SIGTSTP (Ctrl+Z)   → suspends foreground process GROUP
      └── SIGHUP  (terminal close) → sent to session leader → propagates

Shell's job table:
      ├── Job 1 [running]  → foreground (has terminal control)
      ├── Job 2 [stopped]  → suspended (SIGTSTP received)
      └── Job 3 [running]  → background (no terminal input)

Process Groups:
  Every job = a process group (PGID)
  fg/bg operate on the ENTIRE process group, not one PID
  Ctrl+C kills ALL processes in the foreground process group
```

🔷 Core job control commands

```
# ── Start background jobs ──────────────────────────────────────
sleep 300 &
# [1] 12345     ← [job_number] PID

make build &    # [1] 12345
make test &     # [2] 12346

# ── jobs — list current shell's jobs ──────────────────────────
jobs
# [1]   Running    make build &
# [2]-  Running    make test &
# [3]+  Running    rsync -av . /bak/ &
#  + = current job (most recently touched)
#  - = previous job

jobs -l    # Include PIDs
jobs -r    # Only running jobs
jobs -s    # Only stopped jobs
```

```
# ── Ctrl+Z — suspend foreground process ──────────────────────────
vim myfile.txt
^Z
# [1]+  Stopped    vim myfile.txt
# Shell prompt returns — vim is paused in memory, file is safe

# ── fg — bring job to foreground ─────────────────────────────────
fg          # Bring the + (current) job to foreground
fg %1       # Bring job 1 to foreground
fg %vim     # Bring job whose command starts with "vim"
fg %-       # Bring the previous (-) job to foreground

# ── bg — resume stopped job in background ────────────────────────
./long_compile.sh     # Oops, forgot &, now blocking
^Z                    # Suspend it
bg %1                 # Resume it in background
# [1]+ ./long_compile.sh &

# ── wait — synchronize on background jobs ────────────────────────
./step1.sh &
./step2.sh &
wait                  # Block until ALL finish
echo "All done"

# Capture exit status:
./risky.sh &
JOB_PID=$!
wait $JOB_PID
EXIT_CODE=$?
[[ $EXIT_CODE -ne 0 ]] && echo "Failed: $EXIT_CODE"
```

🔷 nohup — Survive terminal disconnect

```
# Problem: closing terminal sends SIGHUP → kills background jobs
# nohup: makes process IGNORE SIGHUP

nohup ./long_running_script.sh &
# nohup: ignoring input and appending output to 'nohup.out'

# What nohup does:
# 1. Sets SIGHUP to SIG_IGN in the child process
# 2. Redirects stdin from /dev/null
# 3. Redirects stdout to nohup.out if stdout is a terminal

# Always specify your own log file:
nohup ./script.sh > /var/log/myscript.log 2>&1 &
nohup python3 server.py --port 8080 > server.log 2>&1 &

# nohup vs screen/tmux:
# nohup  → fire-and-forget scripts, no interaction needed
```

```
# screen → reattachable terminal sessions, interactive work
# tmux   → modern screen, split panes, preferred today
```

🔷 `disown` — Remove job from shell's control

```
# Problem: process already running, forgot nohup, can't restart
# disown removes it from shell's job table
# → shell won't send SIGHUP when it exits

./my_server.py &   # [1] 12345

disown %1          # By job number
disown 12345       # By PID
disown -a          # Disown ALL jobs

# -h: keep in job table but won't get SIGHUP on logout
disown -h %1

# Verify it's gone from job table:
jobs               # No longer listed
ps aux | grep my_server.py   # Still running!

# Rescue workflow (forgot nohup):
./big_job.sh &     # Started without nohup
disown -h %%       # Mark to not receive SIGHUP
# Safe to close terminal now

# nohup vs disown:
# nohup:  changes signal handling IN THE PROCESS (more robust)
# disown: changes what THE SHELL does on exit (rescue option)
```

🔷 Short crisp interview answer

> "Job control uses process groups and signals. Ctrl+Z sends SIGTSTP to the foreground process group, suspending it. `bg` sends SIGCONT to resume it in the background. `fg` gives it back terminal control. `nohup` makes a process ignore SIGHUP — which terminals send to all jobs when closing. `disown` removes a job from the shell's job table so the shell won't send SIGHUP, but the process itself can still receive it. For production scripts I always use `nohup cmd > log 2>&1 &`. If I forget, `disown -h %1` is the rescue."

⚠ Gotchas

```
# GOTCHA 1: Background jobs still write to terminal
./noisy_script.sh &   # Output mixes with your prompt!
# Fix: ./noisy_script.sh > /tmp/output.log 2>&1 &
```

```
# GOTCHA 2: Ctrl+C kills the ENTIRE foreground process group
# If process spawned 5 children, Ctrl+C kills all 5

# GOTCHA 3: jobs are LOCAL to each shell session
# Terminal 1: ./job.sh &
# Terminal 2: jobs → EMPTY! (different shell, different job table)

# GOTCHA 4: Background jobs die if shell exits without disown/nohup
./long.sh &     # Background
exit            # Shell exits → SIGHUP → long.sh dies!
# Fix: nohup ./long.sh & OR ./long.sh & then disown

# GOTCHA 5: wait exit code
./maybe_fails.sh &
wait $!
echo "Exit: $?"    # Captures actual exit code of background job
```

2.3 `top` and `htop` — Reading Load Average, CPU, Memory Columns

🔷 What they are

`top` and `htop` are **real-time process monitors** — they refresh every 1-3 seconds showing the live state of CPU, memory, and processes. `top` is installed everywhere; `htop` is the friendlier, more feature-rich successor.

🔷 `top` — Dissecting the output

```
top - 14:32:15 up 12 days, 3:45, 2 users, load average: 1.23, 0.87, 0.65
Tasks: 156 total,  1 running, 154 sleeping,  0 stopped,  1 zombie
%Cpu(s):  8.3 us,  2.1 sy,  0.0 ni, 88.9 id,  0.5 wa,  0.0 hi,  0.2 si
MiB Mem :  15983.4 total,   1204.5 free,  10245.2 used,   4533.7
buff/cache
MiB Swap:   2048.0 total,   1998.3 free,     49.7 used.   5200.3 avail Mem

PID    USER   PR  NI   VIRT    RES    SHR  S  %CPU  %MEM  TIME+    COMMAND
12345  nginx  20   0   512m   128m    32m  S   8.3   0.8   1:23.45  nginx
67890  java  -20 -20   4.2g   3.1g    45m  S  12.1  19.8  45:12.33  java
  999  root   20   0    65m    12m     8m  R   0.3   0.1   0:02.11  top
```

🔷 Load average — the most important line

```
# load average: 1.23, 0.87, 0.65
#               ────  ────  ────
#               1min  5min  15min

# Load average = average number of processes WANTING the CPU
# (running + waiting for CPU) over the time period

# For a SINGLE-CORE machine:
# load = 1.0 → CPU 100% utilized
# load = 0.5 → CPU 50% utilized
# load = 2.0 → overloaded, 1 process waiting on average

# For MULTI-CORE (e.g., 4 cores):
# load = 4.0 → 100% utilized (1 process per core)
# load = 8.0 → overloaded (2 processes per core)

# Find number of cores:
nproc
grep -c ^processor /proc/cpuinfo

# Rule: load/core < 1.0 → fine | = 1.0 → at capacity | > 1.0 → overloaded

# Reading the TREND (most important):
# 1.23, 0.87, 0.65 → load was low, is RISING  → investigate
# 1.23, 1.45, 1.87 → load was high, FALLING   → recovering
# 1.23, 1.21, 1.25 → load STABLE              → chronic issue

# ⚠ High load ≠ always high CPU!
# Load includes processes waiting for DISK I/O (D state)
# You can have load=10 with CPU=5% if everything is I/O bound
```

🔷 CPU line breakdown

```
# %Cpu(s): 8.3 us, 2.1 sy, 0.0 ni, 88.9 id, 0.5 wa, 0.0 hi, 0.2 si, 0.0 st

# us = user space     — your applications consuming CPU
# sy = system/kernel  — kernel doing work on behalf of processes
# ni = nice           — user processes with modified nice values
# id = idle           — CPU doing nothing (want this HIGH)
# wa = iowait         — CPU idle but WAITING for I/O to complete
# hi = hardware IRQ   — handling hardware interrupts
# si = software IRQ   — handling software interrupts
# st = steal          — time stolen by hypervisor (VM overhead)

# What to look for:
# High us → app is CPU-hungry → find and optimize the process
# High sy → lots of syscalls  → excessive I/O or context switching
# High wa → I/O bottleneck    → check disk with iostat, iotop
# High st → VM throttled      → host overloaded, check cloud CPU credits
```

```
# Low  id → CPU under pressure → need more CPUs or optimization

# Press '1' in top → expands to show each CPU core individually
# %Cpu0  : 82.3 us ...   ← this core is maxed
# %Cpu1  :  4.1 us ...   ← this core is idle
# Useful for spotting single-threaded bottlenecks pinned to one core
```

🔷 Memory lines

```
# MiB Mem : 15983.4 total, 1204.5 free, 10245.2 used, 4533.7 buff/cache
# MiB Swap:  2048.0 total, 1998.3 free,    49.7 used. 5200.3 avail Mem

# total      = physical RAM installed
# free       = RAM not used at all (usually intentionally low)
# used       = RAM used by processes
# buff/cache = RAM used for disk buffers and file cache (RECLAIMABLE)
# avail Mem  = RAM available for new processes (free + reclaimable cache)

# ⚠ CRITICAL: "free" being low is NOT a problem!
# Linux deliberately uses free RAM for disk cache (makes reads faster)
# Kernel reclaims cache instantly when a process needs RAM
# Low "free" with high "avail Mem" = HEALTHY, working as designed

# Real memory pressure indicators:
# avail Mem is LOW AND swap used is HIGH → actual memory pressure
# Swap used INCREASING over time         → memory leak or undersized RAM

# free command gives the same info more clearly:
free -h
#               total    used    free   shared  buff/cache  available
# Mem:           15Gi    10Gi   1.2Gi    256Mi       4.5Gi       5.1Gi
# Swap:         2.0Gi    49Mi   2.0Gi
```

🔷 Process columns in top

```
# PID    USER  PR  NI   VIRT    RES    SHR  S  %CPU  %MEM  TIME+   COMMAND

# PR  = priority (kernel scheduling priority, lower number = higher
priority)
# NI  = nice value (-20 to +19, lower = gets more CPU)
# VIRT = virtual memory — all mapped address space (misleading, usually
huge)
# RES  = resident memory — physical RAM actually in use ← THE one that
matters
# SHR  = shared memory — shared libraries, shared mappings
# S    = state: R(running) S(sleeping) D(disk wait) Z(zombie) T(stopped)
# %CPU = CPU this sample (can exceed 100% for multi-threaded — normal!)
```

```
# TIME+= cumulative CPU time (hours:minutes:seconds.hundredths)

# RES - SHR = private memory unique to this process
```

🔷 `top` keyboard shortcuts

```
# While top is running:
1           → per-CPU view (show each core individually)
M           → sort by memory (RES)
P           → sort by CPU %
T           → sort by TIME+
k           → kill a process (prompts for PID and signal)
r           → renice a process
u           → filter by user
f           → field management (add/remove/reorder columns)
d           → change refresh interval
H           → toggle thread view
V           → tree/forest view
c           → toggle full command path
i           → hide idle processes (0% CPU)
q           → quit
Shift+W     → save layout to ~/.toprc

# Non-interactive (for scripts):
top -b -n 1                  # -b=batch mode, -n 1=one iteration
top -b -n 1 | head -20       # Get header + top processes
top -b -n 1 -o %CPU | head -20   # Sort by CPU
```

🔷 `htop` — The modern monitor

```
# Install
apt install htop    # Debian/Ubuntu
yum install htop    # RHEL/CentOS

# htop advantages over top:
# - Color coded per column type
# - Mouse clickable headers (click to sort)
# - Per-core CPU bars (visual, instant)
# - Horizontal scrolling (no truncation)
# - Multi-select processes (Spacebar)
# - Kill multiple at once

# htop header:
# CPU bars: [|||||||       50%]  ← visual bar per core
# Mem bar:  [||||||||||||||8.5G]  ← visual bar for RAM
# Swp bar:  [|                ]   ← swap usage
# Tasks: 156, Load: 1.23 0.87 0.65, Uptime: 12 days
```

```
# Key shortcuts:
F1 or ?     → help
F2          → setup (columns, colors, meters)
F3 or /     → search process by name
F4          → filter (show only matching)
F5          → tree view toggle
F6          → sort by column
F7 / F8     → decrease / increase nice value
F9          → kill (shows signal menu)
F10 or q    → quit
Space       → tag/select a process
k           → kill tagged processes
e           → show environment variables
l           → lsof for selected process (open files)
s           → strace selected process
H           → hide user threads
K           → hide kernel threads
```

🔷 Worked example: diagnosing a slow server

```
# Scenario 1 — disk bottleneck:
# load average: 14.23, 12.87, 9.65   ← rising on a 4-core machine
# %Cpu(s):  2.3 us,  1.1 sy,  0.0 ni, 12.0 id, 84.3 wa
#                                                 ──────────
#                                                 84% iowait!
# Diagnosis: CPU is idle, waiting for disk → disk is the bottleneck
# Next steps: iostat -x 1, iotop

# Scenario 2 — CPU hog:
# load average: 8.23, 7.87, 7.65    ← consistently high on 4-core
# %Cpu(s): 95.3 us,  3.1 sy,  0.0 ni, 0.4 id, 0.5 wa
#          ──────────────────────────────────────────
#          95% user CPU! Something is spinning in userspace
# Next steps: sort top by %CPU → find the PID → strace / perf

# Scenario 3 — memory pressure:
# MiB Swap: 2048 total, 12 free, 2036 used.  102.3 avail Mem
#                       ────────────────     ───────────────
#                       Swap nearly full!    But avail Mem is 102MB —
tight
# Is swap growing? Run: watch -n 5 free -h
# If swap used growing → memory leak in an application
```

🔷 Short crisp interview answer

> "The most important line in top is load average — average processes wanting CPU over 1/5/15 minutes. On a 4-core machine, load 4.0 = 100% utilized. I compare 1-min vs 15-min to see trend

> direction. For CPU, I focus on `wa` — high iowait with low user CPU means disk bottleneck, not CPU. For memory, 'free' being low is meaningless since Linux uses free RAM for cache. I look at `avail Mem` and watch swap usage trend. In htop I use F3 to search, F5 for tree view, Space+F9 to kill multiple processes, and press '1' in top to see individual CPU cores."

⚠ Gotchas

```
# GOTCHA 1: Load includes I/O wait, not just CPU
# load = 10, CPU = 5% → disk bottleneck, not CPU
# Check iowait % in the CPU line to confirm

# GOTCHA 2: %CPU > 100% is NORMAL for multi-threaded processes
# A 4-thread process maxing all threads shows ~400% CPU
# This is correct — it's using 4 full cores

# GOTCHA 3: "free" memory being low is NOT a problem
# Linux caches disk reads in free RAM → "free" looks low
# Look at "avail Mem" for actual headroom

# GOTCHA 4: top -b output is for scripts, not interactive
# %CPU in batch mode is since last sample, may look jumpy
# Take multiple samples and average them for reliable data

# GOTCHA 5: Load in containers can be misleading
# Containers share the HOST kernel's load average
# A container limited to 1 CPU may show host load of 20
# Use cgroup CPU metrics for container-accurate monitoring
```

2.4 Signals & `kill` — SIGTERM, SIGKILL, SIGHUP, SIGINT, Trapping Signals

🔷 What signals are

Signals are **software interrupts** — the kernel's way of notifying a process that an event has occurred. They're asynchronous: a process can be doing anything when a signal arrives, and execution jumps to the signal handler immediately.

🔷 How signals work internally

```
Signal lifecycle:
  Sender                    Kernel                    Receiver
    │                          │                          │
    │── kill(pid, SIGTERM) ───►│                          │
```

```
    |                           | marks signal as pending  |
    |                           | in receiver's task_struct|
    |                           |                          |
    |                           |── delivers signal ──────►|
    |                           |   (on return from        |  signal
handler runs
    |                           |    syscall or interrupt)  |  OR default
action
    |                           |                          |

Signal dispositions (what a process can do with a signal):
  1. Default action   (kernel defined — terminate, dump core, stop,
ignore)
  2. Custom handler   (process installs its own function)
  3. Ignore           (SIG_IGN — signal is silently discarded)
  4. Block            (signal is held pending until unblocked)

⚠ SIGKILL and SIGSTOP CANNOT be caught, blocked, or ignored!
   Only the kernel handles them — always.
```

🔷 The important signals

```
Signal     Num  Default    Description
──────────────────────────────────────────────────────────────
SIGHUP      1   Terminate  Terminal hangup / daemon config reload
SIGINT      2   Terminate  Ctrl+C — keyboard interrupt
SIGQUIT     3   Core dump  Ctrl+\ — quit with core dump
SIGKILL     9   Terminate  CANNOT be caught/ignored — instant kill
SIGTERM    15   Terminate  Graceful termination request (catchable)
SIGSTOP    17   Stop       CANNOT be caught/ignored — pause process
SIGTSTP    20   Stop       Ctrl+Z — terminal stop (CAN be caught)
SIGCONT    18   Continue   Resume a stopped process
SIGCHLD    17   Ignore     Child process stopped or terminated
SIGUSR1    10   Terminate  User-defined signal 1 (app-specific)
SIGUSR2    12   Terminate  User-defined signal 2 (app-specific)
SIGALRM    14   Terminate  Timer signal (from alarm() syscall)
SIGPIPE    13   Terminate  Broken pipe (write to closed read-end)
SIGSEGV    11   Core dump  Segmentation fault (invalid memory access)
SIGBUS      7   Core dump  Bus error (misaligned memory access)

# List all signals:
kill -l
trap -l    # In bash
```

🔷 `kill` — Sending signals

```
# kill sends a signal to a process
# Despite the name, it's a SIGNAL SENDER — not just for killing!

# Default signal = SIGTERM (15) — graceful shutdown request
kill 12345           # Send SIGTERM to PID 12345
kill -15 12345       # Same — explicit
kill -SIGTERM 12345  # Same — by name

# SIGKILL — guaranteed termination (no cleanup)
kill -9 12345
kill -SIGKILL 12345

# SIGHUP — reload config (convention for many daemons)
kill -HUP 12345
kill -1 12345

# SIGUSR1 / SIGUSR2 — user-defined (app-specific behavior)
kill -USR1 12345     # nginx: reopen log files
kill -USR2 12345     # nginx: graceful upgrade

# ── Targeting by name with pkill ──────────────────────────────
pkill nginx            # SIGTERM to all processes named "nginx"
pkill -9 nginx         # SIGKILL all "nginx"
pkill -HUP nginx       # Reload all nginx processes
pkill -u alice         # Kill all processes owned by alice
pkill -f "python3 app" # Match against full command line (-f)

# ── Targeting with killall ────────────────────────────────────
killall nginx          # SIGTERM to all processes named exactly "nginx"
killall -9 nginx
killall -HUP nginx

# pkill vs killall:
# pkill: regex match on name, supports -u (user), -g (group), -f (full
cmd)
# killall: exact name match, simpler but less flexible

# ── Kill a process group ──────────────────────────────────────
kill -- -12345    # Negative PID = send to process GROUP 12345
# Kills the process AND all its children in the same process group

# ── pgrep — find PIDs by name (no killing) ────────────────────
pgrep nginx        # Print PIDs of all nginx processes
pgrep -u alice     # PIDs of all alice's processes
pgrep -a nginx     # Print PID AND command line
pgrep -l nginx     # Print PID AND process name

# Use in scripts:
NGINX_PID=$(pgrep -x nginx | head -1)
kill -HUP "$NGINX_PID"
```

🔷 SIGTERM vs SIGKILL — the critical distinction

```
# SIGTERM (15) — the RIGHT way to stop a process
# - Process receives the signal
# - Custom handler can: flush buffers, close connections, clean temp files
# - Process controls its own shutdown
# - Allows graceful shutdown

kill -15 my_app    # Send SIGTERM — please shut down cleanly

# SIGKILL (9) — the nuclear option
# - Kernel terminates the process IMMEDIATELY
# - No cleanup possible — buffers unflushed, connections dropped
# - File corruption risk if writing to disk
# - Zombie left behind until parent calls wait()

kill -9 my_app     # Only use when SIGTERM doesn't work!

# PRODUCTION RULE: Always try SIGTERM first, then wait, then SIGKILL
graceful_kill() {
    local pid=$1
    local timeout=${2:-30}

    kill -SIGTERM "$pid" 2>/dev/null || return 0   # Send SIGTERM
    local elapsed=0
    while kill -0 "$pid" 2>/dev/null; do           # kill -0 = check
existence
        if (( elapsed >= timeout )); then
            echo "SIGTERM timeout after ${timeout}s, sending SIGKILL"
            kill -SIGKILL "$pid" 2>/dev/null
            return
        fi
        sleep 1
        (( elapsed++ ))
    done
    echo "Process $pid exited cleanly"
}

# kill -0 checks if process EXISTS (sends no signal)
kill -0 12345 2>/dev/null && echo "process exists" || echo "process gone"
```

🔷 Trapping signals in Bash scripts

```
# trap — install signal handler in bash script
# Syntax: trap 'command' SIGNAL [SIGNAL...]

# ── Cleanup on exit ──────────────────────────────────────────────
TMPFILE=$(mktemp)
trap 'rm -f "$TMPFILE"' EXIT    # Runs on ANY exit (normal, error, signal)
```

```bash
# EXIT is a pseudo-signal — fires when script exits for ANY reason
# This is the most important trap pattern — always use for cleanup

# ── Handle Ctrl+C gracefully ──────────────────────────────────────
trap 'echo "Interrupted — cleaning up..."; exit 130' INT
# Exit 130 = 128 + 2 (SIGINT=2) — convention for signal-interrupted exit

# ── Full production script with signal handling ──────────────────
#!/usr/bin/env bash
set -euo pipefail

TMPDIR_WORK=$(mktemp -d)
LOCKFILE=/var/run/myjob.lock
exec 9>"$LOCKFILE"
flock -n 9 || { echo "Already running!"; exit 1; }

cleanup() {
    echo "Cleaning up..."
    rm -rf "$TMPDIR_WORK"
    rm -f "$LOCKFILE"
}

handle_error() {
    echo "Error at line $1" >&2
    cleanup
    exit 1
}

trap 'cleanup' EXIT
trap 'echo "Caught SIGINT"; exit 130' INT
trap 'echo "Caught SIGTERM"; exit 143' TERM
trap 'handle_error $LINENO' ERR

# Main script work here...

# ── SIGHUP for config reload ──────────────────────────────────────
# Daemons use SIGHUP for live config reload (nginx, sshd, etc.)
# In a bash daemon:
CONFIG_FILE=/etc/myapp/config.conf
load_config() { source "$CONFIG_FILE"; }
load_config  # Initial load
trap 'echo "Reloading config..."; load_config' HUP

# ── Trapping for graceful shutdown in a loop ──────────────────────
RUNNING=true
trap 'echo "Shutting down..."; RUNNING=false' TERM INT

while $RUNNING; do
    # Do work...
    sleep 1
done
echo "Exited cleanly"
```

```
# ── ignore a signal ──────────────────────────────────────────
trap '' HUP    # Ignore SIGHUP (like nohup does)
trap '' PIPE   # Ignore SIGPIPE (common for pipelines)

# ── Reset signal to default ──────────────────────────────────
trap - HUP     # Reset SIGHUP to default behavior

# ── List current traps ───────────────────────────────────────
trap -p        # Print all current trap settings
```

🔷 Short crisp interview answer

> "Signals are asynchronous software interrupts from the kernel to a process. `kill` sends them — despite the name it's a signal sender, not just for killing. SIGTERM (15) is the polite request: the process catches it, cleans up, and exits. SIGKILL (9) is instant kernel-level termination — no handler runs, no cleanup. Always SIGTERM first, wait, then SIGKILL. In scripts, `trap 'cleanup' EXIT` is essential — it fires on any exit including signals. SIGHUP is conventionally used for daemon config reload — `kill -HUP nginx_pid` reloads nginx without downtime. `kill -0 pid` checks process existence without sending a real signal."

⚠ Gotchas

```
# GOTCHA 1: SIGKILL leaves a zombie until parent calls wait()
# Process is killed but PID entry stays in process table
# Parent must call wait() to clean up the zombie entry

# GOTCHA 2: kill -9 can corrupt data
# Databases, write-ahead logs, in-memory buffers — all unflushed
# Use SIGTERM and wait for clean exit whenever possible

# GOTCHA 3: kill sends to PID, not name — verify PID first!
# kill 1234    ← are you SURE 1234 is the process you think it is?
# ps -p 1234   ← verify before killing
# pkill nginx  ← safer — sends to all processes named nginx

# GOTCHA 4: trap EXIT vs trap ERR vs set -e interaction
# set -e makes script exit on error
# trap ERR fires before set -e exits
# trap EXIT fires AFTER ERR handler
# Order: ERR handler → set -e exit → EXIT handler

# GOTCHA 5: Subshells inherit traps in a modified way
# Subshells RESET signals to default (trap handlers don't transfer)
# Functions inherit the parent's traps
( trap 'echo hi' EXIT; exit )   # Does print "hi" — subshell runs handler
```

2.5 Process States — R, S, D, Z, T — What Each Means in Production

🔷 What process states are

The kernel tracks the state of every process — whether it's actively using CPU, waiting for something, or stopped. Understanding states is critical for diagnosing production issues: zombie processes, D-state hangs, and excessive sleeping processes all mean very different things.

🔷 The state machine

```
          fork()
            │
            ▼
        CREATED
            │
            ▼
     ┌──── RUNNING (R) ──────────────────────────┐
     │   actively using CPU                      │
     │   OR ready, waiting for CPU slice         │
     └───────────────────────────────────────────┘
        │                    │                    │
        │                    │                    │
        │ I/O syscall        │ Ctrl+Z /           │ exit()
        │ lock wait          │ SIGSTOP            │
        ▼                    ▼                    ▼
INTERRUPTIBLE        STOPPED (T)           ZOMBIE (Z)
 SLEEP (S)           (paused,               (dead, waiting
 (waiting,            awaiting               for parent
  woken by            SIGCONT)               to wait())
  signal or event)
        │
        │ disk I/O
        │ kernel lock
        ▼
UNINTERRUPTIBLE
  SLEEP (D)
  (cannot be woken
   by signals —
   not even SIGKILL)
```

🔷 State R — Running

```
# R = Running OR Runnable (ready to run, waiting for CPU)
# Both "actually on CPU now" and "ready, waiting for scheduler" are R
```

```
ps aux | grep " R "
# PID   %CPU  STAT  COMMAND
# 12345  99.9   R   [crypto thread]   ← actively running
# 12346  45.2   R+  bash              ← R+ means foreground

# STAT modifiers (second letter):
# + = foreground process group
# s = session leader
# l = multi-threaded (uses CLONE_THREAD)
# N = low priority (nice > 0)
# < = high priority (nice < 0)

# High R count = CPU contention
ps aux | awk '$8 ~ /^R/ {count++} END {print "R state:", count}'

# One persistently R process = CPU hog
# Many R processes = CPU is oversubscribed
```

🔷 State S — Interruptible Sleep

```
# S = Sleeping, waiting for an event or I/O
# MOST processes are in S state — this is completely normal
# Can be woken by: a signal, an event, I/O completion

# Examples:
# bash waiting for keyboard input → S
# nginx worker waiting for HTTP request → S
# sleep 100 → S (waiting for timer)

ps aux | awk '$8 == "S" {count++} END {print "S state:", count}'
# Typical healthy system: 90%+ of processes are S

# Why S is important:
# Process in S CAN receive and respond to signals (SIGTERM, etc.)
# If you kill -15 a sleeping process, it wakes up and handles the signal
```

🔷 State D — Uninterruptible Sleep (THE dangerous one)

```
# D = "Disk sleep" — waiting for I/O that CANNOT be interrupted
# Process is deep in a kernel code path handling disk/network I/O
# CANNOT receive signals — not even SIGKILL!

# Causes:
# - Waiting for disk read/write to complete
# - Waiting for NFS mount to respond
# - Waiting for a kernel lock
# - Stuck on a hung/slow storage device
```

```
# How to spot it:
ps aux | awk '$8 ~ /^D/ {count++} END {print "D state:", count}'
ps aux | grep " D "

# D for a brief moment = normal (I/O in progress)
# D for minutes/hours  = PROBLEM (hung I/O, stuck NFS, bad disk)

# What to do with D state processes:
# 1. Identify what they're waiting for
cat /proc/12345/wchan    # Shows kernel function they're blocking in
# "nfs_file_read" → NFS issue
# "jbd2_journal_commit_transaction" → disk journal flush
# "mutex_lock" → kernel lock contention

# 2. Check for NFS issues
mount | grep nfs
df -h    # Hangs on NFS = mounted NFS is unresponsive

# 3. Check I/O wait
iostat -x 1    # Look for high await (ms per I/O operation)

# 4. If it's a disk issue:
dmesg | tail -50    # Kernel I/O error messages

# ⚠ You CANNOT kill a D state process — SIGKILL is ignored!
# The process will die on its own when the I/O completes or times out
# If it never completes: reboot is the only option
```

🔷 State Z — Zombie

```
# Z = Zombie — process has exited but parent hasn't called wait()
# The process is DEAD — no memory, no resources, not running
# BUT: it holds a PID and an entry in the process table
#      (to preserve the exit code for the parent to read)

# Zombies look like:
ps aux | grep Z
# 12345  parent  20  0  0  0  0  Z  ...  [dead_process] <defunct>

# A few zombies = completely normal (transient, cleaned up quickly)
# Many zombies = parent has a bug (not calling wait())
# 1000s of zombies = eventually exhausts PID space → system instability

# How to find zombies and their parents:
ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/ {print "zombie:", $0}'
# Get the PPID — that's the buggy parent
# Sending SIGCHLD to parent may trigger it to call wait():
kill -CHLD <parent_pid>
```

```
# If parent is stuck or buggy → kill the parent
# When parent dies → zombies are re-parented to PID 1 (systemd)
# Systemd calls wait() immediately → zombies cleaned up

# You CANNOT kill a zombie directly — it's already dead
kill -9 <zombie_pid>   # Silently ignored — zombie is already dead
```

🔷 State T — Stopped

```
# T = Stopped — process received SIGSTOP or SIGTSTP (Ctrl+Z)
# Process is frozen in place, not running, not waiting for I/O
# Can be resumed with SIGCONT

# T state in ps:
ps aux | grep " T "
# Shows: STAT = T  or  T+

# How processes get to T:
# Ctrl+Z in terminal → SIGTSTP → T state
# kill -STOP pid     → SIGSTOP → T state (cannot be caught)
# Debugger attaches  → SIGSTOP → T state

# Resume with:
kill -CONT 12345    # Send SIGCONT
fg %1               # Or from shell job control

# Special substate in newer kernels:
# t (lowercase) = traced/stopped by debugger (strace, gdb attached)
```

🔷 Production diagnosis table

```
STAT   | Meaning                | Normal?   | Action if unexpected
-------|------------------------|-----------|-----------------------------
R      | Running/Runnable       | Yes       | If many R: check load, CPU
S      | Interruptible sleep    | Yes (90%)| Normal — waiting for events
D      | Uninterruptible sleep  | Brief OK  | If minutes+: check disk/NFS
Z      | Zombie                 | Transient| If many: fix parent's wait()
T      | Stopped                | Debugging| kill -CONT or fg to resume

Key production scenarios:
  Load high, CPU low, many D → disk/NFS bottleneck
  Load high, CPU high, many R → CPU oversubscribed
  Load OK, many Z growing    → parent process bug (missing wait)
  Process won't die          → check if it's in D state
```

🔷 Short crisp interview answer

> "Linux processes have 5 main states. R is running or runnable — on CPU or queued. S is interruptible sleep — normal for processes waiting for I/O or events; can receive signals. D is uninterruptible sleep — blocked deep in kernel waiting for I/O; CANNOT receive signals including SIGKILL. Z is zombie — process exited but parent hasn't called wait() to collect the exit status; it holds a PID but no resources. T is stopped via SIGSTOP/SIGTSTP. In production, D state hanging for minutes means NFS or disk problems; many Z means a parent process bug not calling wait(); many R means CPU oversubscription."

⚠ Gotchas

```
# GOTCHA 1: You cannot kill a D state process
kill -9 <D_state_pid>   # IGNORED — SIGKILL cannot interrupt D state
# The process will exit when the I/O completes or times out
# Only option if it never recovers: reboot

# GOTCHA 2: You cannot kill a zombie (it's already dead)
kill -9 <zombie_pid>   # Process is dead, kill does nothing
# Kill the PARENT to clean up zombies

# GOTCHA 3: High zombie count → PID exhaustion
cat /proc/sys/kernel/pid_max   # Maximum PIDs (default 32768 or 4194304)
# 1000s of zombies can exhaust this, preventing new processes

# GOTCHA 4: D state can show 100% CPU wait in top
# Process not running but system load is high
# iowait in %Cpu line will be high

# GOTCHA 5: R includes "ready to run" — not just "on CPU"
# A machine with 100 R-state processes doesn't have 100 CPUs
# Scheduler picks one at a time — others wait their turn
```

2.6 `nice` & `renice` — CPU Scheduling Priority

🔷 What niceness is

"Niceness" is a hint to the Linux scheduler about how much CPU time a process should get relative to others. A **lower nice value = higher priority** (less "nice" to other processes). A **higher nice value = lower priority** (more "nice" — yields CPU to others).

🔷 How it works internally

```
Nice values:
  -20 ──────────────────────── 0 ──────────────────────── +19
  Highest priority        Default        Lowest priority
  (least nice to others)             (most nice to others)

The CFS scheduler converts nice values to CPU time weights:
  nice -20 → weight 88761  (gets ~10x more CPU than nice 0)
  nice   0 → weight  1024  (default)
  nice  19 → weight    15  (gets ~1/15 of nice 0's CPU)

This is RELATIVE, not absolute:
  If only one process is running, it gets 100% CPU regardless of nice
  Nice matters only when COMPETING for CPU
```

🔷 `nice` — Start a process with adjusted priority

```
# Syntax: nice -n VALUE command
# Default nice value: 0
# Only root can set NEGATIVE nice values (higher priority)

# Start a process with lower priority (positive nice)
nice -n 10 ./cpu_intensive_job.sh    # lower priority
nice -n 19 ./batch_job.sh           # lowest priority
nice ./background_task.sh           # default +10 (nice without -n)

# Start a process with higher priority (negative nice — root only)
sudo nice -n -10 ./critical_service.sh   # higher priority
sudo nice -n -20 ./realtime_task.sh      # highest priority

# Check current nice value of a process:
ps -p 12345 -o pid,ni,cmd
# PID  NI  CMD
# 12345  10  ./cpu_intensive_job.sh

# Common use cases:
# Compile jobs during business hours (don't disturb users)
nice -n 15 make -j8

# Batch data processing (run at low priority)
nice -n 19 python3 process_dataset.py

# Backup jobs (don't impact production)
nice -n 15 rsync -av /data /backup
```

🔷 `renice` — Change priority of running process

```
# Syntax: renice -n VALUE -p PID
# Syntax: renice -n VALUE -u USER

# Lower priority (any user can make OWN processes nicer)
renice -n 10 -p 12345    # Change PID 12345 to nice 10
renice -n 15 -p 12345    # Change to nice 15

# ⚠ Non-root users can ONLY increase nice (make lower priority)
# Non-root CANNOT decrease nice (cannot raise priority)
renice -n 5 -p 12345    # OK if process is currently nice 10 or higher?
NO!
# Actually: non-root can ONLY go from current value UPWARD (more positive)

# Root can set any value in any direction:
sudo renice -n -5 -p 12345   # Increase priority
sudo renice -n 19 -p 12345   # Set to lowest priority

# Renice all processes of a user:
renice -n 10 -u www-data    # Lower all www-data processes

# Renice a process group:
renice -n 5 -g 12345        # All processes in group 12345

# Dynamic niceness with top:
# In top: press 'r' → enter PID → enter new nice value

# In htop:
# Select process → F7 (decrease nice = higher priority)
#                  F8 (increase nice = lower priority)
```

🔷 Real-world patterns

```
# Pattern 1: CPU-intensive batch job that shouldn't impact production
#!/usr/bin/env bash
# At the top of heavy batch scripts:
renice -n 15 -p $$       # Renice THIS script's process
ionice -c 3 -p $$        # Also lower I/O priority (idle class)

# Pattern 2: Automated backup at low priority
0 2 * * * nice -n 19 ionice -c 3 /usr/local/bin/backup.sh >>
/var/log/backup.log 2>&1

# Pattern 3: Emergency - critical process getting starved
# Your database is being starved by a rogue process
ps aux | grep " R " | head    # Find the CPU hogs
sudo renice -n 19 -p <hog_pid>    # Demote it immediately
sudo renice -n -5  -p <db_pid>    # Promote the DB

# Pattern 4: Check nice values in top
```

```
# NI column in top shows nice values
# PR column = actual kernel priority = 20 + nice value
# PR 20 = nice 0 (default)
# PR 39 = nice 19 (lowest user priority)
# PR -20 to -2 = kernel/rt priorities

# ionice — I/O scheduling priority (separate from CPU nice)
ionice -c 1 -n 0 ./realtime    # Realtime I/O class (root only)
ionice -c 2 -n 7 ./batch       # Best-effort, lowest priority
ionice -c 3 ./background        # Idle class — only when nothing else
needs disk
ionice -p 12345                 # Check current I/O class of process
```

🔷 Short crisp interview answer

> "Nice values range from -20 (highest CPU priority) to +19 (lowest). The default is 0. The scheduler gives more CPU time slices to lower nice values when processes compete for CPU. `nice -n 15 ./job.sh` starts a job at reduced priority. `renice -n 10 -p PID` adjusts a running process. Only root can set negative nice values (raise priority). In production I use nice+19 for batch jobs, backups, and compiles to prevent them impacting interactive or production workloads. `ionice` handles I/O scheduling separately — the idle class (`-c 3`) makes a process do I/O only when the disk is otherwise idle."

⚠ Gotchas

```
# GOTCHA 1: Non-root cannot raise priority (decrease nice)
renice -n 5 -p 12345   # If current is nice 10 → FAILS for non-root!
# "setpriority: Permission denied"
# Only root can go to lower (more negative) nice values

# GOTCHA 2: Nice only matters when processes COMPETE for CPU
# If you're the only process using CPU, nice 19 still gets 100%
# Nice matters during CPU contention — otherwise irrelevant

# GOTCHA 3: nice -n vs nice without -n
nice -n 10 cmd    # Set nice to +10
nice cmd          # ALSO sets to +10 (default adjustment is +10)
nice -10 cmd      # BSD syntax — sets nice to -10! (confusing)

# GOTCHA 4: PR vs NI in top
# PR = 20 + NI for normal processes (user-visible priority)
# PR values < 0 are real-time priorities (not nice-based)
# Don't confuse PR with NI — renice changes NI not PR directly

# GOTCHA 5: nice affects process AND its children
nice -n 15 ./script.sh
# ALL processes that script spawns also run at nice 15
# This is usually what you want for batch jobs
```

2.7 `/proc/PID` Internals — `cmdline`, `maps`, `fd`, `status`, `limits`

🔷 What /proc is

`/proc` is a **virtual filesystem** — it has no files on disk. The kernel generates every file's content on-the-fly when you read it. `/proc/PID/` gives you a real-time window into every aspect of a running process without needing special tools.

🔷 /proc/PID layout

```
/proc/12345/
├── cmdline       ← full command line (null-separated)
├── comm          ← process name (short, 15 chars max)
├── cwd           ← symlink to current working directory
├── environ       ← environment variables (null-separated)
├── exe           ← symlink to the executable binary
├── fd/           ← directory of symlinks to open file descriptors
│   ├── 0         → /dev/pts/0  (stdin)
│   ├── 1         → /dev/pts/0  (stdout)
│   ├── 2         → /dev/pts/0  (stderr)
│   ├── 3         → /var/log/app.log
│   └── 4         → socket:[12345]
├── fdinfo/       ← detailed info about each file descriptor
├── maps          ← memory mappings (virtual address space layout)
├── smaps         ← detailed memory stats per mapping
├── mem           ← process memory (readable with ptrace)
├── net/          ← network stats from this process's view
├── ns/           ← namespace symlinks (net, mnt, pid, etc.)
├── stat          ← process status (machine-readable, used by ps/top)
├── status        ← process status (human-readable)
├── limits        ← resource limits (ulimit values)
├── io            ← I/O stats (bytes read/written)
├── wchan         ← kernel function process is sleeping in
├── stack         ← kernel stack trace
└── oom_score     ← OOM killer score (higher = killed first)
```

🔷 `cmdline` — Full command

```
# /proc/PID/cmdline: full command, args separated by null bytes (\0)
cat /proc/12345/cmdline
# python3/usr/local/bin/app.py--port8080--host0.0.0.0   ← looks garbled
```

```
# Readable version:
cat /proc/12345/cmdline | tr '\0' ' '
# python3 /usr/local/bin/app.py --port 8080 --host 0.0.0.0

# Or use xargs:
xargs -0 < /proc/12345/cmdline

# Why use /proc instead of ps?
# ps truncates at terminal width; /proc never truncates
# Useful for: finding exact args of long-running processes

# comm — short name (15 chars, what ps shows in COMM column)
cat /proc/12345/comm
# python3

# exe — the actual binary
ls -la /proc/12345/exe
# /proc/12345/exe -> /usr/bin/python3
```

🔷 `status` — Human-readable process info

```
cat /proc/12345/status
# Name:    python3
# Umask:   0022
# State:   S (sleeping)
# Tgid:    12345        ← Thread Group ID (= PID for single-threaded)
# Ngid:    0
# Pid:     12345
# PPid:    9876         ← Parent PID
# TracerPid: 0          ← 0 = not being traced; non-zero = debugger
attached
# Uid:     1000  1000  1000  1000    ← real, effective, saved, filesystem
UIDs
# Gid:     1000  1000  1000  1000
# FDSize:  64           ← current size of fd table
# Groups:  1000 4 24 27 ...
# VmPeak:  524288 kB    ← peak virtual memory
# VmSize:  512344 kB    ← current virtual memory
# VmRSS:   131072 kB    ← physical RAM (RSS)
# VmData:   98765 kB    ← data segment size
# VmStk:     8192 kB    ← stack size
# VmExe:      512 kB    ← text (code) size
# VmLib:    65536 kB    ← shared library size
# Threads: 8            ← number of threads
# SigBlk:  0000000000000000   ← blocked signals bitmask
# SigIgn:  0000000000001000   ← ignored signals bitmask
# SigCgt:  0000000180014a03   ← caught signals bitmask

# Quick checks:
```

```
grep VmRSS /proc/12345/status   # RSS memory
grep Threads /proc/12345/status # Thread count
grep PPid /proc/12345/status    # Parent PID
grep TracerPid /proc/12345/status  # Is it being debugged?
grep State /proc/12345/status   # Current state
```

🔷 `fd/` — Open file descriptors

```
# List all open file descriptors
ls -la /proc/12345/fd/
# lrwxrwxrwx  0 -> /dev/pts/0        (stdin)
# lrwxrwxrwx  1 -> /dev/pts/0        (stdout)
# lrwxrwxrwx  2 -> /dev/pts/0        (stderr)
# lrwxrwxrwx  3 -> /var/log/app.log
# lrwxrwxrwx  4 -> socket:[98765]    (TCP connection)
# lrwxrwxrwx  5 -> /tmp/myapp.pid
# lrwxrwxrwx  6 -> /dev/null
# lrwxrwxrwx  7 -> anon_inode:[eventfd]

# Count open FDs (fd leak detection):
ls /proc/12345/fd | wc -l
# Compare to ulimit:
grep "open files" /proc/12345/limits

# Socket FD details:
# socket:[98765] → look up 98765 in /proc/net/tcp (inode number)
cat /proc/net/tcp | grep $(printf '%x\n' 98765)

# The deleted file trick:
# If a file is deleted but a process still has it open:
ls -la /proc/12345/fd/ | grep deleted
# lrwxrwxrwx  3 -> /var/log/old.log (deleted)
# The file space won't be freed until the FD is closed!
# This is the classic "disk full but no big files found" scenario
lsof +L1    # Find all open-but-deleted files (lsof approach)

# Production: watch for FD count growing (leak indicator)
watch -n 5 'ls /proc/12345/fd | wc -l'
```

🔷 `maps` — Memory layout

```
cat /proc/12345/maps
# 55a1b3200000-55a1b3400000 r-xp  00000000  fd:01  123456
/usr/bin/python3
#
──────────────────────────────────────────────────────────────────────
# start-end addr  perms     offset   dev     inode  pathname
```

```
#
# Permissions: r=read, w=write, x=execute, p=private, s=shared
#
# 55a1b3600000-55a1b3800000 r--p  00000000  fd:01  123456
/usr/bin/python3
# 55a1b3800000-55a1b3900000 rw-p  00000000  fd:01  123456
/usr/bin/python3
# 7f8a1c000000-7f8a1c200000 rw-p  00000000  00:00       0  [heap]
# 7f8b34000000-7f8b34200000 r-xp  00000000  fd:01  654321
/lib/x86_64/libc.so.6
# 7ffdf3a00000-7ffdf3c00000 rw-p  00000000  00:00       0  [stack]
# 7ffdf3e00000-7ffdf3e01000 r-xp  00000000  00:00       0  [vdso]

# Sections you'll see:
# [heap]    = heap memory (malloc'd data)
# [stack]   = thread stack
# [vdso]    = virtual dynamic shared object (fast syscalls)
# [vsyscall]= legacy fast syscall mapping
# anon      = anonymous mappings (mmap'd without file)

# Detect library injection / unexpected mappings:
grep -v '\.so\|heap\|stack\|vdso\|vsyscall' /proc/12345/maps | grep -v
python
```

🔷 `limits` — Resource limits

```
cat /proc/12345/limits
# Limit                     Soft Limit  Hard Limit  Units
# Max cpu time              unlimited   unlimited   seconds
# Max file size             unlimited   unlimited   bytes
# Max data size             unlimited   unlimited   bytes
# Max stack size            8388608     unlimited   bytes
# Max core file size        0           unlimited   bytes
# Max resident set          unlimited   unlimited   bytes
# Max processes             63484       63484       processes
# Max open files            1024        65536       files
# Max locked memory         65536       65536       bytes
# Max address space         unlimited   unlimited   bytes
# Max file locks            unlimited   unlimited   locks
# Max pending signals       63484       63484       signals
# Max msgqueue size         819200      819200      bytes
# Max nice priority         0           0
# Max realtime priority     0           0

# KEY limits to monitor:
# Max open files = 1024 soft  ← classic bottleneck for high-connection
apps
# Max processes             ← limits threads too (each thread = a task)

# Current FD usage vs limit:
```

```
FD_USED=$(ls /proc/12345/fd | wc -l)
FD_LIMIT=$(awk '/Max open files/ {print $4}' /proc/12345/limits)
echo "FDs: $FD_USED / $FD_LIMIT"

# Change limits for running process (requires root or prlimit):
prlimit --nofile=65536:65536 --pid 12345    # Raise FD limit live
prlimit --pid 12345                          # Show current limits
```

🔷 Other useful /proc/PID files

```
# wchan — what kernel function is the process sleeping in
cat /proc/12345/wchan
# wait_woken          ← waiting for network data (normal)
# nfs_file_read       ← stuck in NFS read (problem if long)
# futex_wait_queue_me ← waiting on a mutex/lock

# io — I/O statistics
cat /proc/12345/io
# rchar: 1234567    ← bytes read (including cached)
# wchar: 987654     ← bytes written (including cached)
# syscr: 1234       ← read() syscall count
# syscw: 567        ← write() syscall count
# read_bytes: 65536    ← actual disk bytes read
# write_bytes: 32768   ← actual disk bytes written

# oom_score — OOM killer score (0-1000, higher = killed first)
cat /proc/12345/oom_score
# 234

# oom_score_adj — manually adjust OOM score (-1000 to +1000)
echo -1000 > /proc/12345/oom_score_adj   # Never kill this process (root
only)
echo  1000 > /proc/12345/oom_score_adj   # Kill this first when OOM

# environ — environment variables
cat /proc/12345/environ | tr '\0' '\n'
# PATH=/usr/local/bin:/usr/bin:/bin
# HOME=/home/alice
# ...

# cwd and exe
ls -la /proc/12345/cwd    # What directory the process is in
ls -la /proc/12345/exe    # What binary is running
```

🔷 Short crisp interview answer

> "`/proc/PID` is a virtual filesystem the kernel generates on-the-fly — there's nothing on disk. The most useful files: `cmdline` gives the full untruncated command (args null-separated, use `tr '\0'`

> ' '); `fd/` lists all open file descriptors as symlinks — counting them detects FD leaks; `status` gives readable state, RSS, PPID, and signal masks; `limits` shows ulimit values including max open files; `maps` shows the virtual memory layout; `wchan` shows what kernel function the process is sleeping in, which is invaluable for diagnosing D-state hangs. I use `ls /proc/PID/fd | wc -l` to detect FD leaks and `grep VmRSS /proc/PID/status` to check actual memory use."

⚠ Gotchas

```
# GOTCHA 1: /proc files must be read atomically — they're not real files
# Don't: cat /proc/12345/status | process_slowly
# The content changes between reads — use a single cat/read

# GOTCHA 2: Reading /proc requires appropriate permissions
# /proc/12345/maps, /proc/12345/mem → require ptrace permissions
# Root can read all; regular users can only read their own processes'
details

# GOTCHA 3: FD count in /proc/PID/fd includes the fd/ directory read
itself
ls /proc/12345/fd | wc -l   # May count 1 extra (the ls process's own fd)
# More accurate: lsof -p 12345 | wc -l

# GOTCHA 4: deleted files
ls -la /proc/12345/fd/ | grep "(deleted)"
# Space won't be freed until FD is closed
# Classic symptom: df shows full, du shows plenty of space
# Fix: find the process, close the FD (or restart the process)

# GOTCHA 5: /proc/PID disappears when process exits
# If process exits while you're reading /proc/PID: ENOENT errors
# Script defensively: ls /proc/PID/status 2>/dev/null || echo "gone"
```

2.8 Linux Scheduler — CFS, Time Slices, cgroups CPU Limits

🔷 What the scheduler is

The Linux CPU scheduler decides **which runnable process gets the CPU next and for how long**. The Completely Fair Scheduler (CFS) has been the default since kernel 2.6.23. Understanding it explains why processes compete for CPU, how containers limit CPU, and why "nice" values work the way they do.

🔷 CFS — Completely Fair Scheduler

```
Core idea: every process should get a "fair" share of CPU time

CFS tracks "virtual runtime" (vruntime) per process:
  vruntime = actual CPU time weighted by priority
  Lower priority → vruntime increases FASTER (penalty)
  Higher priority → vruntime increases SLOWER (bonus)

The scheduler ALWAYS runs the process with LOWEST vruntime next.

  Process A (nice  0): vruntime = 100ms  ← RUN THIS ONE
  Process B (nice  0): vruntime = 105ms
  Process C (nice 10): vruntime =  80ms  ← but effective vruntime is
higher

Red-Black Tree (sorted by vruntime):

   ┌─────┐
   │ A   │ ← leftmost = lowest vruntime = next to run
   └─────┘
      │
      ├── B
      └── C
          └── D

After running A, its vruntime increases → it moves right in the tree
The new leftmost process becomes the next to run
```

🔷 Time slices and scheduling

```
# CFS doesn't use fixed time slices — it uses a "target latency"
# Target latency: every runnable process should get at least one run
# within this period (default: 6ms per process or 24ms total)

# With 4 processes at equal priority:
# Each gets 24ms / 4 = 6ms per period
# This gives 166 context switches per second per CPU

# The minimum granularity prevents too-small slices:
cat /proc/sys/kernel/sched_min_granularity_ns
# 1000000 (1ms) — no slice smaller than this

cat /proc/sys/kernel/sched_latency_ns
# 6000000 (6ms) — target scheduling period per process

cat /proc/sys/kernel/sched_wakeup_granularity_ns
# 1000000 (1ms) — wakeup preemption granularity

# View per-process scheduling stats:
cat /proc/12345/schedstat
# 1234567890  9876543  1234
# CPU time (ns)  wait time (ns)  timeslices run
```

```
# More detailed scheduler stats:
cat /proc/12345/sched
# se.exec_start              :    1234567890.123456
# se.vruntime                :          123.456789
# se.sum_exec_runtime        :         1234.567890  ← total CPU time in ms
# nr_switches                :               12345  ← context switch count
# nr_voluntary_switches      :               10000  ← yielded voluntarily
# nr_involuntary_switches    :                2345  ← preempted by
scheduler
```

🔷 cgroups — CPU limiting for containers

```
# cgroups (Control Groups) = kernel feature to limit, account, isolate
# resources (CPU, memory, I/O, network) for groups of processes

# cgroups v1 CPU limits (older, still common):
# Located at: /sys/fs/cgroup/cpu/

# Set CPU quota for a process:
# cpu.cfs_period_us = scheduling period (default 100ms = 100000 µs)
# cpu.cfs_quota_us  = how much CPU time the group gets per period
#   quota = -1 → no limit
#   quota = 50000 with period 100000 → 50% of ONE CPU
#   quota = 200000 with period 100000 → 2 full CPUs (200%)

# Create a cgroup and limit CPU:
sudo mkdir /sys/fs/cgroup/cpu/myapp
echo 100000 > /sys/fs/cgroup/cpu/myapp/cpu.cfs_period_us  # 100ms period
echo 50000  > /sys/fs/cgroup/cpu/myapp/cpu.cfs_quota_us   # 50% CPU limit
echo 12345  > /sys/fs/cgroup/cpu/myapp/tasks               # Add PID to
group

# cgroups v2 (modern, unified hierarchy):
# Located at: /sys/fs/cgroup/
# All resource types in one hierarchy

# Check what cgroup a process is in:
cat /proc/12345/cgroup
# 12:cpu,cpuacct:/docker/abc123def456
# 11:memory:/docker/abc123def456
# (shows cgroup path per subsystem)

# ── Docker / Kubernetes CPU limits via cgroups ──────────────────────
# Docker: --cpus=0.5 → sets quota to 50000µs per 100000µs period
docker run --cpus=0.5 nginx
# This creates: cpu.cfs_quota_us = 50000

# Kubernetes:
# resources:
```

```
#   limits:
#     cpu: "500m"   → 500 millicores = 0.5 CPU
#                   → cfs_quota_us = 50000

# Check CPU throttling (containers being throttled):
cat /sys/fs/cgroup/cpu/docker/abc123/cpu.stat
# nr_periods       100    ← total scheduling periods
# nr_throttled      15    ← periods where quota was exhausted
# throttled_time 1500000  ← total nanoseconds throttled

# High nr_throttled/nr_periods ratio → container is CPU constrained
# Symptom: application is slow, CPU seems available, but container is
throttled

# ── CPU shares (relative weight, cgroups v1) ──────────────────────
cat /sys/fs/cgroup/cpu/myapp/cpu.shares
# 1024  ← default; higher = more CPU weight (like nice but for cgroups)
echo 2048 > /sys/fs/cgroup/cpu/myapp/cpu.shares  # 2x weight vs default
```

🔷 NUMA awareness and CPU pinning

```
# NUMA (Non-Uniform Memory Access):
# Multi-socket servers have multiple CPU nodes
# Memory access is faster for local node than remote node

# Check NUMA topology:
numactl --hardware

# Pin process to specific CPU(s):
taskset -c 0,1 ./my_program           # Run on CPUs 0 and 1 only
taskset -c 0-3 ./my_program           # Run on CPUs 0 through 3

# Set affinity of running process:
taskset -cp 0,1 12345                 # Pin PID 12345 to CPUs 0,1

# Check current CPU affinity:
taskset -p 12345

# NUMA-aware execution:
numactl --cpunodebind=0 --membind=0 ./my_program  # All on NUMA node 0

# View scheduling history (which CPUs a process used):
cat /proc/12345/status | grep Cpus_allowed
# Cpus_allowed:   ff   ← bitmask; ff = all 8 CPUs allowed
```

🔷 Short crisp interview answer

> "CFS (Completely Fair Scheduler) tracks 'virtual runtime' per process using a red-black tree. The process with the lowest vruntime runs next. Lower nice values slow vruntime accumulation, giving more CPU. Higher nice values accelerate it. There are no fixed time slices — CFS uses a target latency period divided equally among runnable processes. Container CPU limits work via cgroups `cpu.cfs_quota_us` and `cpu.cfs_period_us` — Docker `--cpus=0.5` sets the quota to 50% of one CPU. High `nr_throttled` in `cpu.stat` means the container is being CPU-throttled by cgroups, which explains slow performance despite available CPU on the host."

⚠ Gotchas

```
# GOTCHA 1: cgroup CPU throttling is invisible from inside the container
# Container's `top` shows low CPU usage but app is slow
# Reason: cgroups throttle it without the container seeing "busy" CPU
# Check: cat /sys/fs/cgroup/cpu/.../cpu.stat for nr_throttled

# GOTCHA 2: CPU "limit" ≠ CPU "request" in Kubernetes
# request = guaranteed minimum (sets cpu.shares)
# limit   = hard maximum (sets cfs_quota)
# A pod can burst above request up to limit IF CPU is available

# GOTCHA 3: Context switch overhead
# High nr_involuntary_switches → process being preempted a lot
# Could indicate it's getting CPU-starved (too many competing processes)
# cat /proc/12345/sched to see switch counts

# GOTCHA 4: nice values and cgroup CPU shares interact
# A process with nice -20 inside a cgroup with low cpu.shares
# may still get less CPU than a nice 0 process in a high-shares cgroup
# cgroup limits are enforced BEFORE nice values matter

# GOTCHA 5: CFS target latency scales with process count
# 2 processes: each gets 3ms slices (6ms total / 2)
# 100 processes: each gets 0.24ms slices → LOTS of context switches
# High process count = high scheduling overhead even at low CPU%
```

2.9 Zombie & Orphan Processes — How They Form, How to Clean Them

🔷 Zombies and orphans — the two scenarios

```
Scenario A: Parent exits BEFORE child → ORPHAN
──────────────────────────────────────────────
  Parent: dies (crashes or exits)
       │
```

```
        └── Child: still running
                   → kernel re-parents to PID 1 (systemd/init)
                   → child continues running normally
                   → systemd will call wait() when child exits
                   → no zombie created (systemd handles it)

Scenario B: Child exits BEFORE parent calls wait() → ZOMBIE
──────────────────────────────────────────────────────────────

  Child: exits (calls exit())
       │
       └── kernel marks as zombie (Z state)
           kernel KEEPS the PID entry in process table
           (to preserve exit code for parent)
           kernel sends SIGCHLD to parent

  Parent: SHOULD call wait() to reap the child
         if parent ignores SIGCHLD or never calls wait() →
         → zombie stays in process table FOREVER (until parent exits)
```

🔷 Zombie processes in detail

```
# How to find zombies:
ps aux | grep Z
ps -eo pid,ppid,stat,comm | awk '$3 ~ /Z/ {print}'

# Output:
# PID   PPID  STAT COMM
# 12345  9876   Z  [myapp] <defunct>

# What zombies consume:
# - A PID entry in the process table
# - An entry in the kernel's task_struct
# - NO memory, NO CPU, NO open files
# They hold ONLY a PID and exit status

# Why zombies matter:
# Linux has a maximum PID limit:
cat /proc/sys/kernel/pid_max    # Default: 32768 (or 4194304 on 64-bit)
# If zombie count reaches pid_max → new processes cannot be created!
# "fork: Cannot allocate memory" error — even with plenty of RAM

# Counting zombies:
ps aux | awk '$8 == "Z" {count++} END {print count+0, "zombies"}'

# Cleaning zombies (3 approaches):
# 1. Signal SIGCHLD to parent (nudge it to call wait()):
kill -CHLD <parent_pid>

# 2. Kill the parent (zombies get re-parented to PID 1):
kill -TERM <parent_pid>    # Parent exits → systemd reaps zombies
```

```
# 3. Wait for natural cleanup (zombies only last as long as parent lives)
# When parent exits, all its zombie children are immediately reaped

# Verify zombies cleaned:
ps aux | grep Z | wc -l

# Debugging the root cause (the parent):
# Find the parent of zombies:
ps -eo pid,ppid,stat,comm | awk '$3 ~ /Z/ {print "zombie:", $1, "parent:",
$2}'
# Then inspect the parent:
strace -e wait4,waitpid -p <parent_pid>   # Is it calling wait?
```

🔷 Orphan processes in detail

```
# How orphans form:
# 1. nohup process (parent closes terminal)
# 2. Parent crashes
# 3. Daemon forks and parent intentionally exits

# Find orphans (processes whose parent is PID 1):
ps -eo pid,ppid,comm | awk '$2 == 1 && $3 != "init" && $3 != "systemd"
{print}'
# Most of these are INTENTIONAL daemons — that's normal!
# systemd manages them after adoption

# Daemon double-fork pattern (creates an intentional orphan):
# This is how traditional Unix daemons detach from the terminal:
#!/usr/bin/env bash
# First fork: detach from shell
if [[ $$ -ne 1 ]]; then
    # Fork once
    ./daemon.py &
    CHILD=$!
    # The child will fork again and exit
    wait $CHILD
    exit 0
fi
# The grandchild continues as a true daemon (PPID=1)

# In Python (common pattern):
# import os
# if os.fork(): sys.exit()   # Parent exits
# os.setsid()               # New session
# if os.fork(): sys.exit()   # Intermediate exits
# # Grandchild is now a true orphan daemon, PID 1 adopted it
```

🔷 Production patterns to prevent zombies

```
# Pattern 1: Always wait() for children in scripts
run_parallel_jobs() {
    local pids=()
    for task in "${TASKS[@]}"; do
        ./process_task.sh "$task" &
        pids+=($!)
    done

    # Reap all children:
    local failed=0
    for pid in "${pids[@]}"; do
        wait "$pid" || (( failed++ ))
    done
    return $failed
}

# Pattern 2: Signal-safe child reaping (for long-running bash daemons)
reap_children() {
    local pid exit_code
    while true; do
        # wait -n = wait for any child (bash 4.3+)
        wait -n pid 2>/dev/null || break
        exit_code=$?
        echo "Child $pid exited with $exit_code"
    done
}
trap 'reap_children' CHLD   # Reap on every SIGCHLD

# Pattern 3: In C/Python applications — the right pattern
# Always: waitpid(-1, &status, WNOHANG) in SIGCHLD handler
# Or: use signalfd/epoll to handle SIGCHLD in main event loop

# Pattern 4: Using a subreaper (Linux 3.4+)
# A process can declare itself a "subreaper" via
prctl(PR_SET_CHILD_SUBREAPER)
# Orphaned processes are re-parented to the subreaper instead of PID 1
# Docker uses this — containerd is the subreaper for containers
```

🔷 Short crisp interview answer

> "Zombies form when a child process exits but the parent doesn't call `wait()` to collect its exit status. The kernel keeps the PID entry alive to preserve the exit code. Zombies hold no resources except a PID slot. The danger is PID exhaustion — if you hit `pid_max`, new processes can't be created even with plenty of RAM. To clean zombies: send SIGCHLD to the parent to trigger a wait() call; if that fails, kill the parent — its zombies get re-parented to PID 1 which immediately reaps them. Orphans form when a parent exits before its children — the kernel re-parents orphans to PID 1 automatically, and systemd handles reaping. Orphans keep running normally."

⚠ Gotchas

```
# GOTCHA 1: You CANNOT kill a zombie directly
kill -9 <zombie_pid>   # Ignored — process is already dead
# Kill the PARENT instead

# GOTCHA 2: "Cannot allocate memory" despite having free RAM
# Symptom: fork() fails, ps aux shows thousands of Z-state processes
# Root cause: PID exhaustion from zombie accumulation
# Fix: cat /proc/sys/kernel/pid_max (increase temporarily if needed)
echo 65536 > /proc/sys/kernel/pid_max   # Temporary increase (not
persistent)

# GOTCHA 3: Orphans are NOT always a problem
# Most daemons (nginx, sshd, etc.) are intentional orphans (PPID=1)
# Finding orphans: ps -eo pid,ppid | awk '$2==1' → mostly legitimate

# GOTCHA 4: Zombie parents hiding as normal processes
# Parent is running, not crashed — it just never calls wait()
# Application bug: fork() without corresponding wait() in a loop
# Each loop iteration creates one zombie → accumulates over time

# GOTCHA 5: Docker containers and zombie processes
# PID 1 inside a container (your app) must handle SIGCHLD and call wait()
# If your app doesn't, zombies accumulate inside the container
# Solution: use tini or dumb-init as PID 1 in Docker images
# They handle SIGCHLD properly: CMD ["tini", "--", "myapp"]
```

2.10 `strace` & `ltrace` — Syscall Tracing for Live Debugging

🔷 What they are

`strace` intercepts and records **system calls** — the boundary between user code and the kernel. `ltrace` intercepts **library calls** (libc, etc.). Together they let you see exactly what a process is doing at the lowest level, without modifying it or having source code.

🔷 How strace works internally

```
Normal execution:
  User code → printf() → write() syscall → kernel → disk/terminal

strace intercepts at the syscall boundary:
  User code → printf() → write() syscall
```

```
                                  │
                                  ▼
                          ┌──────────────┐
                          │ strace (via  │ ← uses ptrace() syscall
                          │ ptrace API)  │    to intercept
                          │ logs the call │
                          └──────────────┘
                                  │
                                  │
                              kernel → disk/terminal

⚠ ptrace-based tracing has overhead: 2-10x slowdown
   Use on production CAREFULLY and briefly
   Alternative for production: eBPF tools (bpftrace, strace-like with
perf)
```

🔷 `strace` — System call tracing

```
# ── Trace a new process ──────────────────────────────────────────
strace ls
# execve("/usr/bin/ls", ["ls"], 0x... /* env */) = 0
# brk(NULL)                               = 0x55a1b4000000
# openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
# fstat(3, {st_mode=S_IFREG|0644, st_size=123456, ...}) = 0
# mmap(NULL, 123456, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f8b34000000
# close(3)                                = 0
# ...
# write(1, "file1.txt  file2.txt\n", 21) = 21
# exit_group(0)                           = ?

# ── Attach to running process ────────────────────────────────────
strace -p 12345         # Attach to PID 12345

# ── Key flags ────────────────────────────────────────────────────
# -e trace=SYSCALL  — filter to specific syscalls
# -e trace=file     — all file-related syscalls (open, read, write,
stat...)
# -e trace=network  — all network syscalls (socket, connect, send...)
# -e trace=process  — process management (fork, exec, wait...)
# -e trace=signal   — signal-related syscalls
# -e trace=ipc      — IPC (pipe, socket...)
# -f                — follow forks (trace child processes too)
# -ff               — follow forks, separate output per child
# -t                — timestamp each syscall (seconds)
# -T                — show time SPENT in each syscall
# -c                — count and summarize (no per-call output)
# -o FILE           — write output to file (essential for long traces)
# -s SIZE           — max string length to print (default 32 chars!)
# -v                — verbose (print unabbreviated structs)
# -x                — print all strings as hex

# ── Most useful combinations ─────────────────────────────────────
```

```
# See only file operations (what files is this process touching?)
strace -e trace=file -p 12345

# See only network operations
strace -e trace=network -p 12345

# Performance profiling: which syscalls take the most time?
strace -c -p 12345
# % time     seconds  usecs/call     calls    errors syscall
# 43.21    0.123456           5     23456             futex
# 23.45    0.067890          10      6789       123   read
# 15.67    0.044567           2     22345             write
# ...

# Trace with timestamps (find slow syscalls):
strace -T -e trace=read,write -p 12345
# read(5, "data...", 4096)              = 4096 <0.000123>
# read(5, "data...", 4096)              = 4096 <5.123456>  ← 5 second
read!
# This 5-second read is your bottleneck

# Full string output (don't truncate):
strace -s 4096 -e trace=write -p 12345

# Follow child processes:
strace -f ./script.sh

# Write to file (trace long-running issues):
strace -f -o /tmp/trace.txt -p 12345

# ── Real debugging scenarios ──────────────────────────────────────

# Scenario 1: Process is hanging — what is it waiting for?
strace -p <hung_pid>
# If it shows: futex(0x..., FUTEX_WAIT, ...) = ?  → waiting for a
mutex/lock
# If it shows: read(5, ...)                       → waiting for data on
fd 5
# If it shows: epoll_wait(7, ...)                 → waiting for I/O
events
# If nothing: check /proc/PID/wchan and /proc/PID/stack

# Scenario 2: Process can't find a file — which path is it trying?
strace -e trace=openat,stat -p <pid>
# openat(AT_FDCWD, "/etc/myapp/config.conf", O_RDONLY) = -1 ENOENT
#                  ──────────────────────────────────          ──────
#                  Exact path it tried                        Not found!

# Scenario 3: Network connection failure
strace -e trace=network -p <pid>
# connect(3, {sa_family=AF_INET, sin_port=htons(5432), sin_addr=...}, 16)
= -1 ECONNREFUSED
# Shows: IP, port, and error code
```

```
# Scenario 4: Permission denied — what file?
strace -e trace=file 2>&1 ./myapp | grep "EACCES\|EPERM"
# openat(AT_FDCWD, "/var/lib/myapp/data", O_RDWR) = -1 EACCES (Permission
denied)

# Scenario 5: What is causing high CPU? (find hot loops)
strace -c -p <high_cpu_pid>
# Look for: high call count on gettimeofday, clock_gettime (busy polling)
# Or: futex with high error count (lock contention)
```

🔷 `ltrace` — Library call tracing

```
# ltrace intercepts LIBRARY calls (not syscalls)
# Shows: printf(), malloc(), fopen(), strdup(), etc.

ltrace ./myprogram
# __libc_start_main(0x400abc, 1, 0x..., 0x..., ...) = 0
# malloc(1024)                                        = 0x55a1b4002260
# fopen("/etc/config", "r")                           = 0x55a1b4002310
# fgets(0x55a1b4002320, 256, 0x55a1b4002310)          = "key=value\n"
# printf("Config loaded\n")                           = 14
# free(0x55a1b4002260)                                = <void>

# ltrace vs strace:
# ltrace: shows LIBRARY calls (higher level, more readable)
# strace: shows KERNEL syscalls (lower level, definitive)
# Use ltrace first for app-level debugging
# Use strace when ltrace isn't enough or for kernel-level issues

ltrace -p 12345          # Attach to running process
ltrace -c ./myprogram    # Summary: count and time library calls
ltrace -l libcrypto.so.1.1 ./myprogram   # Only trace specific library

# ── Practical ltrace use ──────────────────────────────────────────
# Find what config files an app reads:
ltrace -e fopen,fread,fclose ./myapp 2>&1 | grep fopen
# fopen("/etc/myapp/config.yaml", "r") = 0x...

# Detect memory issues (malloc/free patterns):
ltrace -e malloc,free,realloc ./myapp 2>&1 | grep -v "free"
# malloc(1024) = 0x...   [without matching free = potential leak]
```

🔷 Short crisp interview answer

> "`strace` traces system calls using the `ptrace` kernel API — it shows every boundary crossing between user code and the kernel. For debugging I use `-e trace=file` to see exactly which file paths a process tries (great for permission and 'file not found' issues), `-e trace=network` for

> connection problems, and `-c` for a performance summary showing which syscalls dominate. `-T` shows time spent per syscall to find slow I/O. When attaching to a hanging process, I run `strace -p PID` with no filters — the first line usually reveals what it's blocked on (futex for locks, read for I/O, epoll_wait for events). `ltrace` intercepts library calls at a higher level, useful for tracing config file loading or malloc patterns. Both have significant overhead — use briefly in production."

⚠ Gotchas

```
# GOTCHA 1: strace has 2-10x performance overhead
# Don't attach to production processes for long periods
# Use -c for brief summary, then detach
# Alternative: perf trace (eBPF-based, much lower overhead)

# GOTCHA 2: Default string truncation is 32 chars
strace ./myapp 2>&1 | grep write
# write(1, "This is my really long messag"..., 50) = 50
#                            ________________
#                            Truncated at 32 chars!
# Fix: strace -s 4096 ...

# GOTCHA 3: ASLR and PIDs change — scripts using grep on strace output
# Process memory addresses change each run — don't rely on them

# GOTCHA 4: strace on multi-threaded processes needs -f
strace ./multithreaded_app     # Only traces main thread!
strace -f ./multithreaded_app  # Traces all threads and children

# GOTCHA 5: ltrace doesn't work with statically linked binaries
# or stripped binaries (no symbol table)
ltrace ./statically-linked-binary   # No output — can't intercept

# GOTCHA 6: strace changes process behavior (Heisenbug)
# ptrace delivery changes signal timing — rare but real
# Some programs check for tracers and behave differently:
cat /proc/12345/status | grep TracerPid   # Non-zero = being traced
```

2.11 `lsof` — Open Files, Socket States, Debugging FD Leaks

🔷 What lsof is

`lsof` stands for **List Open Files**. On Linux, everything is a file — regular files, sockets, pipes, devices, directories. `lsof` shows every open file across every process on the system, making it the ultimate tool for understanding what processes are actually doing.

🔷 lsof output explained

```
lsof -p 12345
# COMMAND  PID     USER  FD   TYPE    DEVICE  SIZE/OFF  NODE  NAME
# python3  12345   alice cwd  DIR     253,1     4096    123
/home/alice/app
# python3  12345   alice rtd  DIR     253,1     4096      2   /
# python3  12345   alice txt  REG     253,1  3670016  98765
/usr/bin/python3
# python3  12345   alice mem  REG     253,1  1234567  87654   /lib/x86_64-
linux-gnu/libc.so
# python3  12345   alice   0u CHR     136,0      0t0      3   /dev/pts/0
# python3  12345   alice   1u CHR     136,0      0t0      3   /dev/pts/0
# python3  12345   alice   2u CHR     136,0      0t0      3   /dev/pts/0
# python3  12345   alice   3r REG     253,1    65536  54321
/etc/myapp/config.yaml
# python3  12345   alice   4w REG     253,1   123456  43210
/var/log/myapp.log
# python3  12345   alice   5u IPv4  987654      0t0    TCP
10.0.0.5:8080->1.2.3.4:54321 (ESTABLISHED)
# python3  12345   alice   6u IPv4  876543      0t0    TCP    *:8080
(LISTEN)
# python3  12345   alice   7u unix  0x...       0t0   12345
/tmp/myapp.sock

# FD column meanings:
# cwd  = current working directory
# rtd  = root directory
# txt  = program text (executable binary)
# mem  = memory-mapped file (shared library)
# 0u   = file descriptor 0, mode u=read+write
#        (r=read, w=write, u=read+write)
# 3r   = FD 3, read-only
# 4w   = FD 4, write-only

# TYPE column:
# REG  = regular file
# DIR  = directory
# CHR  = character device (terminal)
# BLK  = block device (disk)
# FIFO = named pipe
# IPv4 = IPv4 socket
# IPv6 = IPv6 socket
# unix = Unix domain socket
```

🔷 Core lsof usage

```
# ── By process ──────────────────────────────────────────────────────
lsof -p 12345              # All FDs for PID 12345
```

```
lsof -p 12345,67890        # Multiple PIDs

# Count open FDs for a process:
lsof -p 12345 | wc -l

# ── By user ──────────────────────────────────────────────────────────
lsof -u alice              # All files opened by alice
lsof -u alice -u bob       # alice OR bob
lsof -u ^alice             # All users EXCEPT alice (^ = NOT)

# ── By file / directory ──────────────────────────────────────────────
lsof /var/log/app.log      # Who has this file open?
lsof /var/log/             # Who has any file in this dir open?
lsof +D /var/log/          # Recursively — who has any file under
/var/log?

# ── By port / network ────────────────────────────────────────────────
lsof -i :8080              # What is listening on port 8080?
lsof -i :80 -i :443        # Port 80 or 443
lsof -i TCP                # All TCP connections
lsof -i UDP                # All UDP connections
lsof -i TCP:8080           # TCP specifically on 8080
lsof -i @1.2.3.4           # Connections to/from this IP
lsof -i TCP:1024-65535     # All high ports

# ── By protocol state ────────────────────────────────────────────────
lsof -i TCP -s TCP:LISTEN              # Only listening sockets
lsof -i TCP -s TCP:ESTABLISHED         # Only established connections
lsof -i TCP -s TCP:CLOSE_WAIT          # Only CLOSE_WAIT (connection
leaks!)
lsof -i TCP -s TCP:TIME_WAIT           # Only TIME_WAIT

# ── lsof -n and -P (performance) ─────────────────────────────────────
# -n = don't resolve hostnames (much faster)
# -P = don't resolve port names (much faster)
lsof -nP -p 12345          # Fast: no name resolution

# ── Useful combinations ──────────────────────────────────────────────
# Find what process is listening on port 80:
lsof -nP -i TCP:80 -s TCP:LISTEN

# Find all sockets opened by nginx:
lsof -c nginx -i

# Who is using a specific file (before you try to delete/move it):
lsof /path/to/file

# All network connections on the system:
lsof -nP -i TCP -i UDP
```

🔷 Debugging FD leaks

```
# ── Detecting a file descriptor leak ──────────────────────────────

# Step 1: Check FD count is growing
watch -n 5 'lsof -p 12345 | wc -l'
# If count grows without bound → FD leak

# Step 2: Check against the FD limit
FD_COUNT=$(lsof -p 12345 | wc -l)
FD_LIMIT=$(cat /proc/12345/limits | awk '/Max open files/ {print $4}')
echo "FDs: $FD_COUNT / $FD_LIMIT"
# If FD_COUNT approaching FD_LIMIT → process will fail soon

# Step 3: Find what's leaking
lsof -p 12345 | sort -k 9 | uniq -c -f 8 | sort -rn | head -20
# Shows which file paths are most frequently open
# Many opens of same socket type → socket leak

# Step 4: Look for deleted-but-open files
lsof -p 12345 | grep deleted
# FD 45w REG 253,1 9999999 1234 /var/log/old.log (deleted)
# File deleted on disk but FD still open → space not freed!

# ── The classic "disk full but du shows nothing" problem ──────────
df -h     # Shows /var is 100% full
du -sh /var/*    # No big files found!
# Root cause: a process has deleted files still open

lsof +L1   # +L1 = show files with link count < 1 (deleted but open)
# COMMAND  PID   USER  FD   TYPE  SIZE       NODE NAME
# java    5678   app   23w  REG   9999999999 1234 /var/log/app.log
(deleted)
# Found it! Java has a 9GB deleted log file open

# Fix: restart the process (closes the FD, OS reclaims space)
# Or: truncate in place if you can't restart:
> /proc/5678/fd/23    # Truncate the open FD to 0 bytes (frees space
immediately!)

# ── Ulimit and raising FD limits ──────────────────────────────────

# Check current limits for your session:
ulimit -n        # Soft limit for open files
ulimit -Hn       # Hard limit for open files

# Raise limit for current session:
ulimit -n 65536

# System-wide default in /etc/security/limits.conf:
# *    soft    nofile    65536
# *    hard    nofile    65536

# Per-service in systemd:
```

```
# [Service]
# LimitNOFILE=65536

# Raise limit for a running process (root, Linux 3.5+):
prlimit --nofile=65536:65536 --pid 12345
```

🔷 Network debugging with lsof

```
# ── Replace ss/netstat with lsof for richer output ──────────────────

# What is on port 8080? (with process name and user)
lsof -nP -i TCP:8080
# COMMAND  PID   USER  FD  TYPE  DEVICE  SIZE/OFF  NODE  NAME
# python3  1234  www   5u  IPv4  567890      0t0   TCP   *:8080 (LISTEN)

# How many connections does nginx have?
lsof -c nginx -i TCP | grep ESTABLISHED | wc -l

# Which remote IPs are connected to my app?
lsof -nP -i TCP:8080 -s TCP:ESTABLISHED | awk '{print $9}' | cut -d: -f1 |
sort | uniq -c | sort -rn

# Find TIME_WAIT connections (connection accumulation check):
lsof -nP -i TCP -s TCP:TIME_WAIT | wc -l

# Who is connecting to Redis (port 6379)?
lsof -nP -i TCP:6379

# See all UNIX domain sockets:
lsof -U
lsof | grep .sock    # Find specific socket files
```

🔷 Short crisp interview answer

> "`lsof` lists every open file descriptor across all processes — and on Linux, everything is a file: regular files, sockets, pipes, devices. `lsof -p PID` shows all FDs of a process. `lsof -i :8080` answers 'what's listening on port 8080' with the process name and user. `lsof -i TCP -s TCP:CLOSE_WAIT` finds socket leaks. For the classic 'disk full but no files found' problem I use `lsof +L1` — it finds deleted files that are still held open by processes; the disk space won't be freed until those FDs close. To detect FD leaks, I watch `lsof -p PID | wc -l` growing over time and compare against `/proc/PID/limits`."

⚠ Gotchas

```
# GOTCHA 1: lsof is slow without -n and -P
lsof -i           # Slow — resolves every hostname and port
lsof -nP -i       # Fast — numeric output
# Always use -n (no hostname lookup) -P (no port name lookup) in scripts

# GOTCHA 2: lsof shows ONE row per FD, not one per file
# A file opened with dup() appears twice
# A mmap'd shared library appears once per mapping
# FD count from lsof includes: txt, mem, cwd, rtd entries — not just
numeric FDs
# Compare against /proc/PID/fd/ for numeric FD count only
ls /proc/12345/fd | wc -l   # Accurate numeric FD count

# GOTCHA 3: lsof needs root for complete information
lsof                  # Shows your own processes + some others
sudo lsof             # Shows ALL processes' ALL file descriptors

# GOTCHA 4: Deleted file space not freed until FD is closed
lsof +L1              # Find deleted-but-open files
# You cannot free the space without closing the FD
# Options: restart the process, or truncate via /proc/PID/fd/N

# GOTCHA 5: lsof +D is recursive but slow
lsof +D /var/log/     # Recursively checks every file — very slow on large
dirs
lsof /var/log/        # Non-recursive — check specific files only (faster)
```

🏆 Category 2 — Complete Mental Model

```
PROCESS MANAGEMENT DECISION TREE
═══════════════════════════════

Problem: Something is wrong with a process

├─ 1. Is the process running?
│      ps aux | grep name
│      pgrep -x name
│
├─ 2. What state is it in?
│      ps -p PID -o stat=
│      R=running  S=sleeping  D=DISK WAIT  Z=zombie  T=stopped
│
├─ 3. What is it doing?
│      strace -p PID          → syscalls it's making
│      lsof -p PID            → files/sockets it has open
│      cat /proc/PID/wchan    → kernel function if sleeping
│      cat /proc/PID/stack    → kernel stack trace
│
```

```
├─ 4. Is it using too much CPU?
│      top / htop             → real-time CPU view
│      ps -eo pid,%cpu --sort=-%cpu | head
│      strace -c -p PID       → which syscalls dominate
│      renice -n 10 -p PID    → lower its priority immediately
│
├─ 5. Is it using too much memory?
│      grep VmRSS /proc/PID/status
│      lsof -p PID | wc -l    → FD count (leak indicator)
│      cat /proc/PID/smaps    → per-mapping memory breakdown
│
├─ 6. Is it creating too many processes/zombies?
│      ps aux | grep Z        → find zombies
│      ps -eo pid,ppid,stat | awk '$3~/Z/{print $2}' → find parent
│      kill -CHLD parent_pid  → nudge parent to wait()
│
└─ 7. Can't kill it?
       kill -15 PID           → SIGTERM first
       sleep 10
       kill -0 PID && kill -9 PID   → SIGKILL if still alive
       cat /proc/PID/wchan    → if D state: wait for I/O or reboot

TOOL QUICK REFERENCE
═══════════════════
ps aux            → snapshot of all processes
ps -ef            → snapshot with PPID column
pstree -p         → visual hierarchy with PIDs
top / htop        → real-time CPU/memory
strace -p PID     → syscall tracing
ltrace -p PID     → library call tracing
lsof -p PID       → open files/sockets
lsof -i :PORT     → what's on a port
lsof +L1          → deleted-but-open files
/proc/PID/status  → state, memory, PPID
/proc/PID/fd/     → open file descriptors
/proc/PID/wchan   → what kernel func it's sleeping in
/proc/PID/limits  → ulimit values
kill -l           → list all signals
kill -0 PID       → check if process exists (no signal sent)
renice -n 10 -p P → lower process priority live
prlimit --nofile=65536 --pid P  → raise FD limit live
```

⚠ Master Gotcha List — Category 2

#	Gotcha	Reality
1	VSZ is the real memory usage	RSS is real; VSZ includes unmapped virtual space
2	kill -9 is the first choice	Always try SIGTERM first; SIGKILL prevents cleanup
3	D-state processes can be killed	SIGKILL is ignored in D state — wait for I/O

#	Gotcha	Reality
4	Zombies waste memory	Zombies use NO memory/CPU — only a PID slot
5	You can kill a zombie	Process is already dead; kill the PARENT
6	High load = high CPU	Load includes D-state (I/O wait) — check iowait %
7	Free memory being low = problem	Linux caches in free RAM — check `avail Mem`
8	Background jobs survive terminal close	Without nohup/disown, SIGHUP kills them
9	Orphans are always problems	Most daemons are intentional orphans (PPID=1)
10	strace output is complete	Default truncates strings at 32 chars; use -s 4096
11	lsof is fast	Always use -n -P flags; lsof without them is very slow
12	FD count from lsof = numeric FDs	lsof includes txt/mem/cwd entries; use /proc/PID/fd/
13	Disk full = find big files	Could be deleted-but-open files; use lsof +L1
14	Nice values affect all workloads	Nice only matters when processes COMPETE for CPU
15	Container CPU = host CPU	cgroup throttling is invisible inside the container

🔥 Top Interview Questions

Q1: What is the difference between SIGTERM and SIGKILL?

> SIGTERM (15) is a polite request — the process can catch it, run cleanup code (flush buffers, close connections, remove temp files), and exit cleanly. SIGKILL (9) is unconditional termination by the kernel — no handler runs, no cleanup, data may be corrupted. Always try SIGTERM first and wait for the process to exit. Only escalate to SIGKILL if SIGTERM doesn't work within a reasonable timeout. `kill -0 PID` checks if a process exists without sending a real signal.

Q2: What is a zombie process and how do you get rid of it?

> A zombie is a process that has exited but whose parent hasn't called `wait()` to collect its exit status. The kernel keeps the PID entry alive to preserve the exit code. Zombies consume no CPU or memory — only a PID slot. You cannot kill a zombie directly (it's already dead). To clean zombies: send SIGCHLD to the parent to trigger a `wait()` call. If that doesn't work, kill the parent — its zombies get re-parented to PID 1, which immediately reaps them. The real fix is to patch the parent application to properly call `wait()` after forking.

Q3: A process is in D state and won't die even with kill -9. Why?

> D (Uninterruptible Sleep) means the process is deep in a kernel code path waiting for I/O to complete — typically disk, NFS, or a kernel lock. Signals, including SIGKILL, cannot be delivered while in D state because the kernel code path cannot be safely interrupted. The process will exit D state when the I/O completes or times out. If it stays D forever, it usually means NFS is hanging or the storage device has a problem. Check `cat /proc/PID/wchan` to see which kernel function it's blocked in, and `dmesg` for I/O errors. The only reliable fix if the I/O never completes is a reboot.

Q4: How do you debug why a process is consuming high CPU?

```
# Step 1: Confirm it's CPU (not I/O)
top -p PID           # Check us% vs wa% in CPU breakdown

# Step 2: Find which thread is hot (if multi-threaded)
top -H -p PID        # Per-thread view
ps -p PID -L -o pid,lwp,%cpu,comm

# Step 3: What syscalls dominate?
strace -c -p PID     # Profile: which syscalls, how many, how long

# Step 4: If it's busy-looping (no blocking syscalls):
perf top -p PID      # Show hot functions (requires perf)

# Step 5: Check if it's supposed to be this busy
cat /proc/PID/sched  # Look at nr_involuntary_switches (being preempted a
lot?)
```

Q5: How does a container CPU limit actually work under the hood?

> Docker `--cpus=0.5` (or Kubernetes `cpu: 500m`) sets cgroup CPU quota: `cpu.cfs_quota_us = 50000` with `cpu.cfs_period_us = 100000`. This means the container's processes can use at most 50ms of CPU per 100ms window. If they try to use more, the kernel throttles them — stops scheduling the container's processes for the remainder of that period. This throttling is invisible inside the container: `top` shows the process apparently running at a low %, not 100%. You can detect throttling by checking `nr_throttled` in `/sys/fs/cgroup/cpu/.../cpu.stat`. High throttle ratios explain "the app is slow but CPU looks fine" in container environments.

Q6: Walk me through how you'd debug a process that appears to be hanging.

```
# Step 1: Confirm it's hanging (not just slow)
ps -p PID -o pid,stat,wchan=   # Check state and what it's waiting for

# Step 2: What state is it in?
# D state → stuck in kernel I/O → check /proc/PID/wchan, dmesg
# S state → sleeping → use strace to see what it's waiting for

# Step 3: Attach strace
strace -p PID    # No filter — first output line tells you what it's
blocked on
# futex() → waiting for a lock (deadlock?)
# read()  → waiting for data (what FD? check lsof -p PID)
# epoll_wait() → event loop, nothing to do (is this normal?)

# Step 4: Check what FDs are involved
lsof -p PID      # What files/sockets are open?

# Step 5: If waiting for network:
ss -tnp | grep PID    # Check TCP connection states
```

```
# Step 6: Kernel stack for D state
cat /proc/PID/stack   # Full kernel call stack
```

Document covers all 11 topics in Category 2: Process Management — from basic PID concepts through advanced scheduler internals, zombie lifecycles, and production syscall debugging.